

SIMATS ENGINEERING

Assessment Tool 1 - Annexure A: Analytical Problem Solving

Course: Artificial Intelligence (CSA17) | CO1 (BL3) | Total Marks: 50

Name: **Krise Monali** | Reg No: **192519111** | Signature: _____

Question 1: Water Jug Problem (goal: exactly 2 gallons in the 4-gallon jug)

Problem formulation

- **State:** (x, y) where x = water in the 4-gallon jug (0..4), y = water in the 3-gallon jug (0..3).
- **Initial state:** (0, 0). **Goal test:** x = 2 (any y).
- **Actions (production rules):** fill 4-jug ($x < 4 \rightarrow (4, y)$); fill 3-jug ($y < 3 \rightarrow (x, 3)$); empty 4-jug on ground ($x > 0 \rightarrow (0, y)$); empty 3-jug ($y > 0 \rightarrow (x, 0)$); pour 3-jug into 4-jug until 4-jug is full or 3-jug is empty; pour 4-jug into 3-jug until 3-jug is full or 4-jug is empty.
- **Path cost:** 1 per action. Search used: Breadth-First Search (guarantees the fewest steps).

Solution (6 steps)

Step	Action	State (x, y)
0	Start	(0, 0)
1	Fill the 3-gallon jug from the pump	(0, 3)
2	Pour 3-gallon jug into 4-gallon jug	(3, 0)
3	Fill the 3-gallon jug again	(3, 3)
4	Pour 3-gallon into 4-gallon until 4-gallon is full (1 gallon moves)	(4, 2)
5	Empty the 4-gallon jug onto the ground	(0, 2)
6	Pour the 2 gallons from the 3-gallon jug into the 4-gallon jug	(2, 0) GOAL

Alternative 6-step path: (0,0) $\rightarrow$ (4,0) $\rightarrow$ (1,3) $\rightarrow$ (1,0) $\rightarrow$ (0,1) $\rightarrow$ (4,1) $\rightarrow$ (2,3). **Interpretation:** Both paths reach exactly 2 gallons in the 4-gallon jug in 6 moves, which is the optimal depth found by BFS. The key idea is that the 3-gallon jug is used to measure the difference ($4 - 3 = 1$, and $3 - 1 = 2$).

Question 2: Mars Rover - PEAS Analysis

i. Percepts

- Camera images (stereo/panoramic) of terrain, rocks and obstacles; LIDAR / proximity readings.
- Spectrometer, X-ray and chemical sensor readings from soil/rock samples; drill and arm force/torque feedback.
- Internal state: battery level, solar panel output, wheel odometry, IMU tilt/orientation, temperature, wheel slip, GPS-like localisation estimates.
- Communication: commands and schedules from Earth, communication window status, dust/weather data.

ii. Environment characteristics

Property	Value	Reason
Observability	Partially observable	Sensors see only a limited range; terrain beneath surface and far away is unknown.

Property	Value	Reason
Agents	Single agent	Mission is carried out by the rover alone (Earth operators give goals, not interfering physically).
Determinism	Stochastic	Wheel slip, sand, dust storms and sensor noise make outcomes uncertain.
Episodic / Sequential	Sequential	Current movement and energy use affect all later decisions.
Static / Dynamic	Dynamic	Wind, dust storms, temperature and lighting change while the rover is deliberating.
Discrete / Continuous	Continuous	Positions, speeds, arm angles and sensor values are continuous.
Known / Unknown	Largely unknown	Terrain and sample composition are being discovered.

iii. Actions

Move forward/backward, turn left/right, stop; navigate to waypoint; deploy and rotate robotic arm; drill / scoop / collect sample; analyse sample with instruments; take photos; store or discard sample; transmit data to Earth; recharge (orient panels to the sun); enter sleep/safe mode; avoid obstacle / reroute.

iv. Performance measure

- **Positive:** number and scientific value of samples correctly collected and analysed; area explored; data successfully transmitted; mission lifetime.
- **Negative (penalties):** collisions, getting stuck, tipping over, instrument damage; excess energy consumption; redundant/duplicate samples; time wasted.
- **Combined measure:** Performance = (scientific value of samples + area covered + data returned) - (energy used + damage + time cost).

v. Suitable agent architecture

A **hybrid, goal-based + utility-based agent with learning**, built as a layered architecture:

- **Reactive layer (simple reflex):** immediate responses such as stop on cliff/tilt detection, obstacle avoidance. These must be fast because communication delay to Earth (4-24 min) rules out remote control.
- **Goal-based layer:** plans routes and task sequences (go to site, drill, analyse) to achieve mission goals, using a model of the environment because it is only partially observable.
- **Utility-based layer:** trades off conflicting objectives (science value vs. energy, risk and time) to choose among alternative sites or actions.
- **Learning element:** improves terrain classification and traversal prediction from experience.

Why: a pure reflex agent cannot plan toward goals or handle partial observability; a pure goal-based agent cannot weigh trade-offs such as battery vs. value. The hybrid gives safety, autonomy and optimisation.

Question 3: 8-Queens Problem Formulation and Search

The given configuration is not a solution because the queen in column 1 shares a diagonal with the queen in column 8, so it violates the 'no two queens attack' constraint (same row, column or diagonal).

Problem components

Component	Incremental formulation (preferred)	Complete-state formulation
States	Any arrangement of 0 to 8 queens on the board with no queen attacking another (one queen per column, placed left to right).	All 8 queens on board, one per column (queen i in column i , row r_i).

Component	Incremental formulation (preferred)	Complete-state formulation
Initial state	Empty board (0 queens).	Any arrangement, e.g. all queens in row 1.
Actions	Add a queen to the leftmost empty column in a row not attacked by existing queens.	Move one queen to another square in its column.
Transition model	Returns board with the new queen added.	Returns board with the queen moved.
Goal test	8 queens on the board, none attacked.	No pair of queens attacking (0 conflicts).
Path cost	Not relevant (only the final configuration matters); 1 per step if needed.	Same.

Search space size by formulation

Formulation	States	Notes
Any 8 queens on 64 squares	about 1.8×10^{14} (64P8)	Naive; hopelessly large.
One queen per column	$8^8 = 16,777,216$	Row/column conflicts still possible.
One queen per row and column (permutations)	$8! = 40,320$	Only diagonal conflicts remain.
Incremental, no attacked placements	2,057	Prunes conflicts early; best choice.

Search strategies explored

- **BFS**: complete, but stores all partial boards at each level - high memory.
- **DFS / backtracking (uninformed)**: goes depth 8 only; backtracks as soon as a column has no safe row. Memory $O(8)$. Best uninformed choice since all solutions lie at depth 8.
- **Iterative deepening**: works but repeats effort; no advantage since depth is known.
- **Local search (informed, for reference)**: hill climbing / min-conflicts on the complete-state formulation; very fast but can stall at local minima.

Backtracking pseudocode:

```

PLACE(col, board):
    if col == 8: return board                # goal reached
    for row in 1..8:
        if SAFE(board, row, col):           # no same row / diagonal
            board[col] = row
            result = PLACE(col + 1, board)
            if result != failure: return result
            board[col] = none                # backtrack
    return failure
SAFE: for each earlier column c: board[c] != row
      and |board[c] - row| != |c - col|

```

Result: A valid solution (row of queen in columns 1-8) is **[1, 5, 8, 6, 3, 7, 2, 4]**. Checked: all rows distinct, and (row - col) and (row + col) values are all distinct, so no diagonal attacks. The 8-queens problem has **92 solutions** in total (12 unique up to rotation/reflection).

Question 4: OLA Cab Booking - Problem-Solving Agent

Type of agent

A **goal-based problem-solving agent with utility-based cab selection** (goal-based agent with a utility function). The goal is *reach destination D from location L*; the cab type (mini, micro, sedan, shared, prime ...) is chosen by maximising a utility that balances fare, waiting time/ETA, comfort and availability according to the customer's preference.

PEAS summary

- **Performance:** reach destination, low fare, short wait, comfort, safety.
- **Environment:** road network, traffic, driver availability, prices (surge) - partially observable, dynamic, stochastic.
- **Actuators:** app screens, booking request, route display, payment. **Sensors:** GPS, user input, traffic and driver-location data.

Problem formulation

- **State:** (current location, cab type selected, booking status). **Initial:** user at pickup location.
- **Actions:** enter destination, select cab type, confirm booking, track cab, ride, pay. **Goal test:** user at destination.
- **Path cost:** weighted sum of fare, time and discomfort.

Pseudocode

```
function OLA-AGENT(percept) returns an action
  persistent: seq (action sequence, initially empty), state, goal, problem

  state <- UPDATE-STATE(state, percept)      # GPS location, user input
  if seq is empty then
    goal <- FORMULATE-GOAL(state)           # reach destination D
    problem <- FORMULATE-PROBLEM(state, goal)
    seq <- BOOK-AND-TRAVEL(problem)
    if seq = failure then return NO-CAB-AVAILABLE
  action <- FIRST(seq); seq <- REST(seq)
  return action

function BOOK-AND-TRAVEL(problem)
  source, dest <- problem.source, problem.destination
  cabs <- {mini, micro, sedan, shared, prime, ...}
  options <- empty list
  for each c in cabs do
    if AVAILABLE(c, source) then
      fare <- ESTIMATE-FARE(c, source, dest)
      eta <- ESTIMATE-ARRIVAL(c, source)
      trip <- ESTIMATE-TRIP-TIME(c, source, dest)
      u <- w1*comfort(c) - w2*fare - w3*(eta + trip)  # utility
      add (c, u, fare, eta) to options
  if options is empty then return failure
  display options to user
  chosen <- user's choice (default: option with maximum u)
  CONFIRM-BOOKING(chosen)
  return [CONFIRM, WAIT-FOR-DRIVER, TRACK-RIDE, REACH(dest), PAY, RATE]
```

Interpretation: The agent formulates a goal, formulates the problem, searches the available cab options, then executes the resulting action sequence (book, track, ride, pay). It re-plans if the booking is cancelled or the driver is unavailable.

Question 5: Uniform Cost Search (S to G)

Graph (directed edges and costs)

Edge	S-A	S-G	A-B	A-C	B-D	C-D	C-G	D-G
Cost	1	12	3	1	3	1	2	3

Method: UCS keeps a priority queue ordered by path cost $g(n)$ and always expands the cheapest node; the goal test is applied when a node is *expanded*, which guarantees optimality. Ties are broken in order of insertion.

Step	Node expanded (g)	Frontier after expansion (node : g, parent)
1	S (0)	A:1 (S), G:12 (S)
2	A (1)	C:2 (A), B:4 (A), G:12 (S)
3	C (2)	D:3 (C), B:4 (A), G:4 (C) [G improved from 12 to 2+2 = 4]
4	D (3)	B:4 (A), G:4 (C) [D->G = 3+3 = 6 > 4, no update]
5	B (4)	G:4 (C) [B->D = 7 > 3, ignored]
6	G (4)	Goal selected: stop

Order of expansion: S, A, C, D, B, G.

Path comparison (verification)

Path	Cost
S -> A -> C -> G	1 + 1 + 2 = 4 (optimal)
S -> A -> C -> D -> G	1 + 1 + 1 + 3 = 6
S -> A -> B -> D -> G	1 + 3 + 3 + 3 = 10
S -> G	12

Result and interpretation: The least-cost route from warehouse S to warehouse G is **S -> A -> C -> G** with a total cost of 4 (fuel + time units). The direct route S -> G costs 12, three times more, so the company saves 8 cost units per delivery by using the multi-hop route. UCS is complete and optimal because all edge costs are positive. Time complexity is $O(b^{1+\text{floor}(C^*/e)})$, where C^* is the optimal cost and e the minimum edge cost.